REINFORCEMENT LEARNING FROM VERIFIABLE REWARDS

WHAT LANGUAGE MODELS CAN LEARN FROM VERIFIED ANSWERS

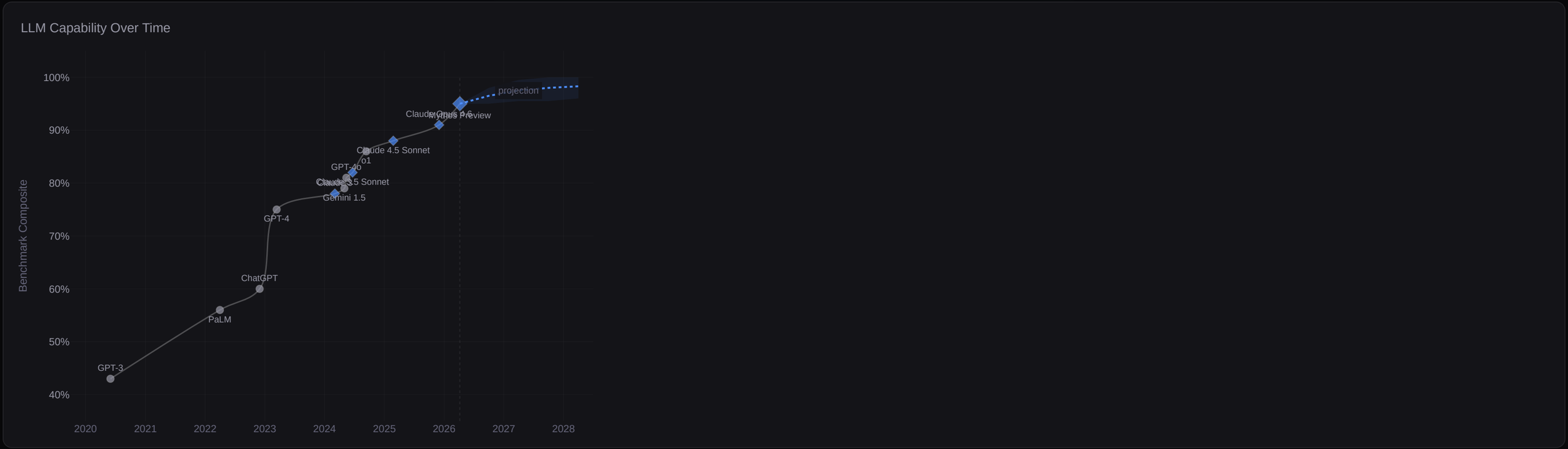
SC-AI/ML Seminar · Florida State University

Tommie Juzek · Lan Li

With a focus on Tulu 3 and OLMo (Allen AI) -- Generated with Opus 4.7

THE RISE OF LARGE LANGUAGE MODELS

Language models have improved at a striking pace — and the trajectory isn't slowing.

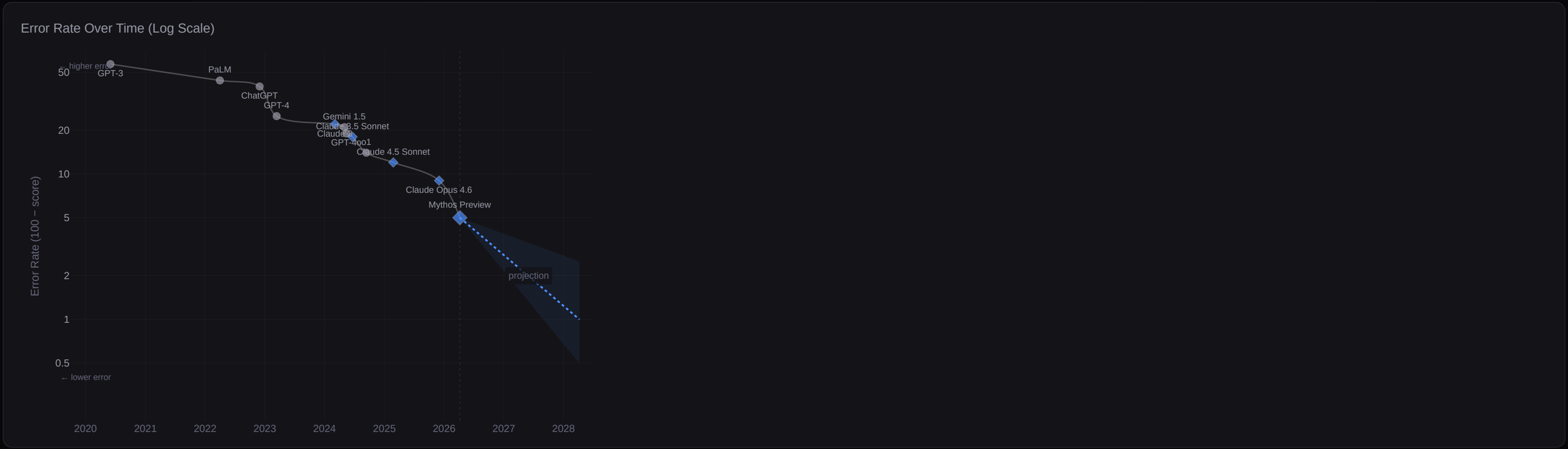


More data, more compute, new architectures, and **new training techniques**.

The question driving this talk: what is the latest technique, and why does it work so well?

THE LAST MILE PROBLEM

Language models have improved at a striking pace — and the trajectory isn't slowing.



Flip the y-axis: plot **error rate** (100 − score) on a log scale, for another perspective.

THE POST-TRAINING PIPELINE

How do we get from a base model to a useful assistant?

Pre-training Learn language → **SFT** Follow instructions → **Preference Tuning** Human preferences → **RLVR** Get the right answer

- **Pre-training**: learn language from trillions of tokens (internet text)
- **SFT** (Supervised Fine-Tuning): learn to follow instructions from curated examples
- **Preference Tuning** (DPO/RLHF): learn which answers humans *prefer*
- **RLVR**: learn to produce *correct* answers on verifiable tasks

The key ingredient of recent progress: **preference learning (PL)**.

- **Human PL**: annotators compare responses, train a reward model (RLHF/DPO)
- **Synthetic PL**: use model-generated preferences — cheaper but risky (self-consuming loops can degrade quality over generations)

Both approaches improved general helpfulness. But as we'll see, they hit a ceiling on precision tasks.

THE PROBLEM WITH RLHF

Traditional alignment requires **human preference data**:

1. Generate two responses to the same prompt
2. Ask a human: *"Which response is better?"*
3. Train a **reward model** on these comparisons, then optimise the LM against it using PPO

WHAT'S WRONG WITH THIS?

- **Expensive**: human labelling at scale costs millions
- **Slow**: feedback loops take weeks
- **Subjective**: different annotators disagree
- **Gameable**: models exploit the reward model without genuine improvement ("reward hacking")
- **Ethically fraught**: annotation is often outsourced to low-paid workers in the Global South under harsh conditions — *Time*'s reporting on OpenAI's Kenyan annotators at Sama (Perrigo, 2023) documented wages under US$2 per hour and sustained exposure to traumatic content.

WHAT PREFERENCE DATA LOOKS LIKE

Every RLHF training example is a human judgment: *"Response A is better than Response B."*

```
#### Chosen (preferred) **Prompt:** *"Explain photosynthesis in one sentence."* > Photosynthesis is the process by which plants convert sunlight, water, and carbon dioxide into glucose and oxygen using chlorophyll.
```

```
#### Rejected **Prompt:** *"Explain photosynthesis in one sentence."* > Plants use the sun to make food and also they release oxygen and stuff which is important for us to breathe.
```

REWARD MODEL TRAINING

- Collect **thousands** of these comparisons
- Train a neural network to better predict human preferences
- Use this learnt reward signal for RL

The reward model is expensive to build, subjective by nature, and gameable.

THE PREFERENCE TUNING PLATEAU

RLHF and DPO made models broadly helpful — but not deeply expert.

General knowledge	Conversation	Math reasoning	Code correctness	Formal reasoning
Above average	Above average	Average	Average	Average

The problem: preference-tuned models learnt to *sound* right, not to *be* right.

They became **(great) jacks of all trades, but masters of none.**

What if we could train on tasks where *correctness is measurable?*

THE RLVR INSIGHT

Some tasks have **objectively verifiable answers**:

TASK	EXAMPLE	VERIFICATION
Math	"What is 234 × 17?" → 3,978	Check the number
Code	"Write a sort function"	Run the test suite
Instruction following	"Write exactly 3 paragraphs"	Count paragraphs

THE IDEA

Replace the **expensive reward model** with a **cheap verification function**:

Verifiable Reward

r = 10 if correct, 0 otherwise

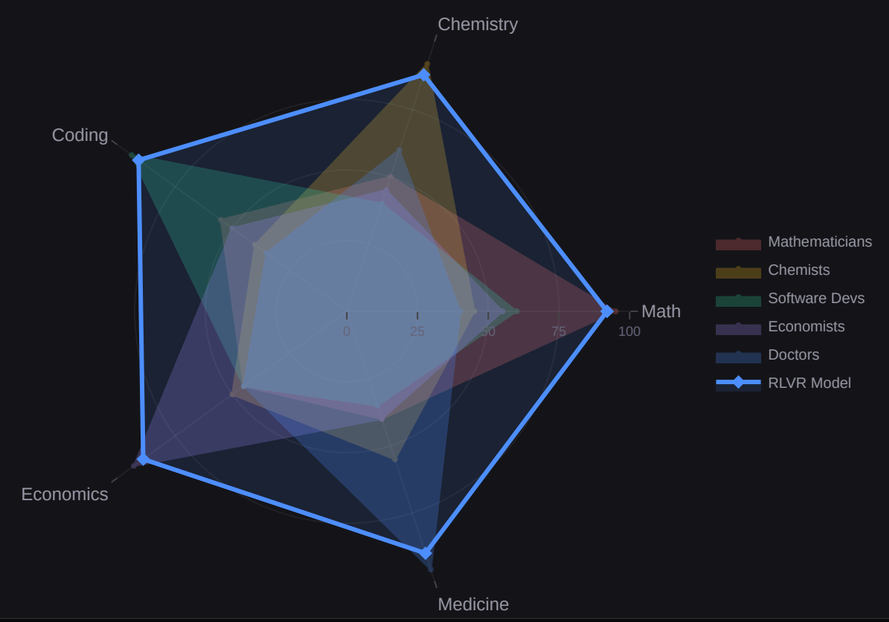
No reward model, no human preferences — just **ground truth** and a function to check it.

This is **RLVR**: Reinforcement Learning from Verifiable Rewards.

FROM DOMAIN EXPERTS TO UNIVERSAL EXPERTISE

THE IDEA *(CONCEPTUAL ILLUSTRATION)*

Domain experts contribute problems with verified answers in their fields:



RLVR combines expert knowledge across domains — producing a model that doesn't just average performance, but **excels in each**.

Illustration is conceptual. Tulu 3 demonstrated this on math and instruction following; the principle extends to any domain with verifiable answers.

WHY THIS MATTERS NOW

RLVR is now part of the training recipe for **all major language models**.

THE OPEN SCIENCE MILESTONE

Tulu 3 (Allen AI, November 2024):

- First fully open RLVR paper — released data, code, models, and full training recipe
- Built on Llama 3.1 (8B, 70B, 405B); competitive with GPT-4o-mini and Claude 3.5 Haiku

THE BROADER LANDSCAPE

- **DeepSeek-R1** (Jan 2025): RLVR at massive scale with emergent chain-of-thought (caveats: distilled from US models, state-backed censorship)
- RLVR principles now appear across the industry

THE TREND

For tasks where ground truth exists, **verifiable rewards are replacing human preferences**. The question: *for which tasks can we build verifiers?*

THE VERIFIABILITY LANDSCAPE

WHERE CAN RLVR HELP — AND WHERE CAN'T IT?

Fully Verifiable - Arithmetic / grade-school math - Competition mathematics - Code generation (test suites) - Format constraints (word count, JSON, case) RLVR potential: **Proven**

Partially Verifiable - Factual QA (knowledge bases) - Logical reasoning (proof checkers) - Translation (reference-based metrics) - Structured data extraction RLVR potential: **Medium** — with caveats

Fundamentally Subjective - Open-ended dialogue / chat - Creative writing / style - Ethical & moral reasoning - Aesthetic judgment RLVR potential: **Minimal** — need human judgment

THEORETICAL LIMITS

- RLVR works best where **cheap, reliable verifiers** exist
- As tasks get harder, the **verifier approaches the difficulty of the task itself**
- Verifying a novel proof requires a proof checker as sophisticated as the prover
- Verifying "good legal advice" requires a lawyer
- The boundary: **aesthetics, taste, ethics, and creativity** cannot be verified algorithmically

THE RLVR TRAINING DATA

The entire dataset is remarkably small: **~30,000 prompts**.

DATASET	COUNT	DOMAIN	VERIFICATION
GSM8K	7,473	Grade school math	Exact match on extracted number
MATH	7,500	Competition math	Exact match (flexible extraction)
IF verifiable	14,973	Instruction following	Programmatic constraint checkers
Total	*29,946*		

CONCRETE EXAMPLES

GSM8K: *"Natalia sold clips to 48 friends in April, then half as many in May. Total?"* → extract last number, check `== 72`

IF constraint: *"Write a poem about autumn. Exactly 4 paragraphs."* → count paragraphs, check `== 4`

Key insight: remarkably small dataset, remarkably simple verifiers.

THE ACTUAL DATA STRUCTURE

This is what RLVR training data looks like on HuggingFace:

GSM8K `allenai/RLVR-GSM` · 7,470 rows

```
{
  "messages": [{
    "role": "user",
    "content": "There are 15 trees
      in the grove. Workers will
      plant trees today. After,
      there will be 21 trees. How
      many did they plant?"
  }],
  "ground_truth": "6",
  "dataset": "gsm8k"
}
```

4 fields. That's it.

MATH `allenai/RLVR-MATH` · 7,500 rows

```
{
  "messages": [{
    "role": "user",
    "content": "Find the domain of
      sqrt(x-2) / sqrt(5-x)"
  }],
  "ground_truth": "[2, 5)",
  "dataset": "MATH",
  "constraint_type": null,
  "constraint": null
}
```

Ground truth is a math expression.

IFEval `allenai/RLVR-IFeval` · 14,973 rows

```
{
  "messages": [{
    "role": "user",
    "content": "Describe IPv6.
      Your entire response should
      be in all lowercase letters."
  }],
  "ground_truth":
    "{\"func_name\":
      \"validate_lowercase\"}",
  "constraint_type":
    "All Lowercase"
}
```

Ground truth is a **function name**.

24 constraint types: lowercase, word count, paragraph count, JSON format, keyword frequency, forbidden words...

Punchline: ~30K JSON rows. Each row is a prompt + a way to check the answer. That's all RLVR needs.

THE VERIFIER FUNCTIONS

The verifiers are trivially simple Python functions:

GSM8K VERIFIER

```python
import re

def verify_gsm8k(response: str, gold_answer: float) -> float:
    """Extract the last number from the response and check it."""
    numbers = re.findall(r'-?[\d,]+\.?\d*', response)
    if not numbers:
        return 0.0  # No number found
    predicted = float(numbers[-1].replace(',', ''))
    return 10.0 if predicted == gold_answer else 0.0
```

IF CONSTRAINT VERIFIER (EXAMPLE: LOWERCASE)

```python
def validate_lowercase(response: str) -> float:
    """Check if the entire response is lowercase."""
    return 10.0 if response == response.lower() else 0.0
```

VERIFIERS IN THE WILD — GSM8K

The previous slide is pedagogical — here's what production verifiers in `allenai/open-instruct` actually look like.

MULTI-STAGE ANSWER EXTRACTION

```python
def extract_gsm_answer(text: str) -> str | None:
    # 1) '#### <answer>'  — GSM8K's native delimiter
    if m := re.search(r"####\s*(-?[\d,]+\.?\d*)", text):
        return m.group(1).replace(",", "")
    # 2) \boxed{…}          — reasoning-model convention
    if m := re.search(r"\\boxed\{([^}]+)\}", text):
        return m.group(1).strip().replace(",", "")
    # 3) Last number anywhere in the response
    nums = re.findall(r"-?[\d,]+\.?\d*", text)
    return nums[-1].replace(",", "") if nums else None

def verify_gsm8k(response: str, gold: str) -> float:
    pred = extract_gsm_answer(response)
    try:
        return 10.0 if pred and float(pred) == float(gold) else 0.0
    except ValueError:
        return 0.0
```

Three fallbacks in order — because models format answers three different ways. Miss any one and a correct answer scores zero. MATH goes further still, using `math_verify` / `sympy` for symbolic equality.

VERIFIERS IN THE WILD — IFEVAL

A REGISTRY OF ~25 PARAMETERISED CONSTRAINT VALIDATORS

```python
class KeywordFrequency(Instruction):
    def __init__(self, keyword: str, frequency: int, relation: str):
        self.keyword, self.frequency, self.relation = keyword, frequency, relation

    def check_following(self, value: str) -> bool:
        n = len(re.findall(rf"\b{re.escape(self.keyword)}\b", value, re.I))
        return n == self.frequency if self.relation == "exactly" else n >= self.frequency

INSTRUCTION_DICT = {
    "keywords:frequency":                    KeywordFrequency,
    "change_case:english_lowercase":         LowercaseInstruction,
    "length_constraints:number_paragraphs": NumberParagraphs,
    # … ~25 more
}

def verify_ifeval(response: str, row: dict) -> float:
    instruction = INSTRUCTION_DICT[row["func_name"]](**row["params"])
    return 10.0 if instruction.check_following(response) else 0.0
```

Each validator is a **class**, not a function — because constraints carry parameters (`keyword`, `frequency`, `relation`). The dispatcher reads `func_name` from the row's `ground_truth` blob, instantiates the right class with the right parameters, and calls `check_following`. Same pattern scales to code verification (add timeout + subprocess sandboxing) and to any new domain you can write a checker for.

THE VERIFIER LANDSCAPE

VERIFIERS COME IN MANY FORMS

DOMAIN	VERIFIER TYPE	EXAMPLE
Math (Tulu 3)	Exact-match	Extract last number, compare to gold
Code (DeepSeek-R1)	Test suite	Run unit tests, pass/fail
Instruction following (Tulu 3)	Constraint check	Count paragraphs, check case

THE CONTRAST WITH RLHF

	RLHF	RLVR
Reward signal	Learnt neural network (~7B params)	5-line Python function
Training data	Thousands of human preference pairs	Just the verification logic
Cost	(human annotators)	~$0 (code)
Failure mode	Reward hacking	Verifier gaming (but more detectable)

THE RLVR OBJECTIVE

THE MATH

$$\max_{\pi_\theta} \mathbb{E}_{y\sim\pi_\theta(x)} \left[v(x,y) - \beta \, \mathrm{KL}\left(\pi_\theta(y|x) \| \pi_{\mathrm{ref}}(y|x)\right)\right]$$

READING IT TERM BY TERM

Reminder: a **policy** π is the LLM viewed as a probability distribution over completions — given a prompt x, it assigns a probability to every possible response y.

- $\max_{\pi_\theta}$ — we're optimising over the policy's weights θ
- $\mathbb{E}_{y\sim\pi_\theta(x)}[\cdot]$ — the **expected value** of $[\cdot]$, averaged over completions y sampled from the **current** policy (on-policy)
- $v(x,y)$ — the **verifiable reward**: $\alpha = 10$ if the verifier passes, 0 otherwise
- $\pi_\theta(y|x)$ — probability the policy being trained assigns to completion y
- $\pi_{\mathrm{ref}}(y|x)$ — same probability under the **frozen reference** (the SFT starting model)
- $\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$ — how far the current policy has drifted from the reference
- β — KL penalty coefficient (≈ 0.05 in Tulu 3) — the dial on the anchor

WHY THE KL PENALTY?

$$\max_{\pi_\theta} \; \mathbb{E}_{y \sim \pi_\theta(x)} \left[v(x, y) - \beta \, \mathrm{KL} \left(\pi_\theta(y|x) \| \pi_{\mathrm{ref}}(y|x) \right) \right]$$

WITHOUT IT, THE MODEL *COLLAPSES*

It finds **degenerate shortcuts** to the reward: always outputting the same number, emitting a memorised correct-looking string, or exploiting a verifier weakness it discovered mid-training. Reward goes up; general capability craters.

WHAT THE ANCHOR DOES

The KL term says: *"be correct, but don't forget everything else you know."* It tethers the policy to the SFT checkpoint — the model that already speaks fluent English, follows instructions, and knows facts about the world. Drifting far from π_{ref} is only worth it if the reward gain justifies the tether tension.

REWARD VS. KL: A TUG OF WAR

FORCE	PULLS TOWARD	WITHOUT IT
$+v(x,y)$	Completions the verifier accepts	No learning signal
$-\beta\,\mathrm{KL}$	The starting model's distribution	Catastrophic forgetting, reward hacking

TUNING B

The two forces are in constant tension throughout training. Tulu 3 picks $\beta \approx 0.05$ — small enough that reward dominates on correct answers, large enough that wandering into off-distribution text gets punished before it pays off.

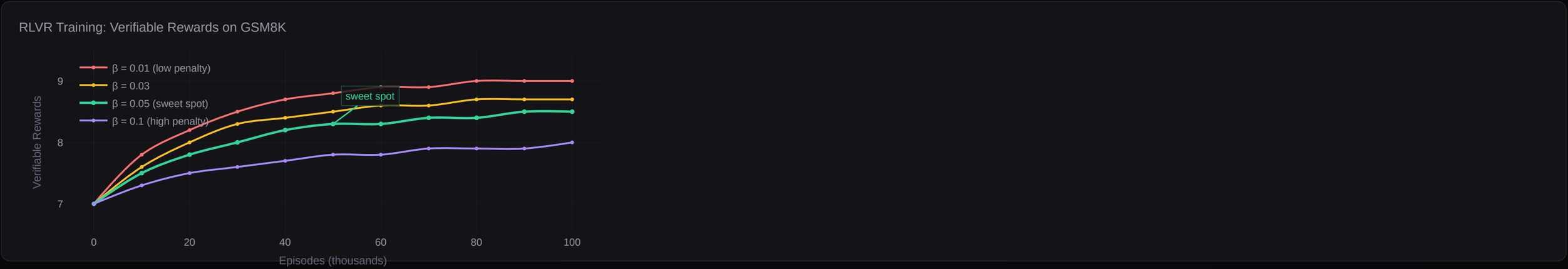
Too small $\Rightarrow$ model collapses to reward hacks. Too large $\Rightarrow$ model refuses to explore and can't learn.

OPTIMISATION

Solved with **PPO** (Schulman et al. 2017) — the clipped surrogate keeps each gradient step small so the tug of war stays stable across thousands of updates. GRPO (DeepSeek, 2024) drops the value network and normalises rewards within each sampled group; same objective, lighter implementation.

TRAINING DYNAMICS

WHAT HAPPENS DURING RLVR TRAINING?



Monitor three signals: **verifiable rewards** (should rise), **KL divergence** (needs control), and **response length** (can inflate as the model hedges).

THE B TRADE-OFF

B (KL PENALTY)	REWARD	KL DIVERGENCE	OVERALL QUALITY
0.01 (low)	Highest	Highest	**Worst** — overoptimisation
0.03	High	Moderate	Good
0.05	**Good**	**Controlled**	**Best**
0.1 (high)	Lowest	Lowest	Moderate — too constrained

Lower β allows higher reward but risks overoptimisation; **higher** β stays close but barely learns. Sweet spot: enough freedom to learn, not enough to forget.

THE TRAINING LOOP

STEP BY STEP

1. **Start** from the DPO checkpoint (not base, not SFT)
2. **Sample** a batch of prompts from the 30K training set
3. **Generate** completions from the current policy π_θ
4. **Verify** each completion with the appropriate verifier
5. **Assign reward**: $r = 10$ if correct, 0 if wrong, -10 if no end-of-sequence token
6. **Update** the policy using PPO with KL penalty against the reference

IMPLEMENTATION DETAILS THAT MATTER

- **Value model** initialised from a general reward model (not the policy)
- **Dropout disabled** during RL (deterministic log-probs for PPO ratio)
- **Non-EOS penalty**: -10 reward if model doesn't produce a stop token
- **~13 epochs** over 30K prompts, checkpointing every 40–100 steps

INFRASTRUCTURE

Asynchronous RL with **vLLM** for inference, **DeepSpeed ZeRO-3** for training. 8B model: ~65h on 8×H100.

RESULTS: WHAT RLVR ACHIEVES

TULU 3 8B (LLAMA 3.1 → DPO → RLVR)

BENCHMARK	LLAMA 3.1	TULU 3 DPO	TULU 3 RLVR	Δ
GSM8K	83.4	84.3	**87.6**	+3.3
MATH	42.5	42.0	**43.7**	+1.7
IFEval	80.6	81.1	**82.4**	+1.3
BigBenchHard	62.8	65.8	**66.0**	+0.2
DROP	61.5	62.5	**62.6**	+0.1
AlpacaEval 2	24.2	33.5	**34.5**	+1.0
Safety	75.2	**87.2**	85.5	–

THE SURPRISE

RLVR was only trained on **math** and **instruction following** — yet it also improved **BigBenchHard**, **DROP**, and **AlpacaEval**. Hypothesis: verifiable tasks teach a general *reasoning discipline* that transfers. At 70B, gains are modest but positive.

KEY DESIGN DECISIONS

WHERE TO START RLVR TRAINING?

STARTING POINT	KL FOR SAME REWARD	FINAL QUALITY
Base model	Huge	Poor
SFT checkpoint	Large	Moderate
DPO checkpoint	**Moderate**	**Best**

Starting from a stronger model (DPO) gives a better launch point — the model already "knows how" to reason; RLVR sharpens it.

PURE VERIFIABLE REWARDS > MIXED SIGNALS

Adding trained reward model scores introduces **noise** and does **not** improve results. **Simpler is genuinely better.**

OVEROPTIMISATION IS REAL

As KL divergence grows, the target task can **improve** while average quality **drops**. Monitor **multiple metrics**, not just the target.

PRACTICAL LIMITATIONS

THINGS THAT GO WRONG

Overoptimisation: the model games the verifier — reward goes up, but overall quality drops (Figure 21 in paper)

Brittle verifiers: exact-match rejects correct answers with different formatting (`42` vs `42.0` vs `\frac{42}{1}`)

Saturation at scale: at 70B, gains are already modest. Does RLVR keep helping as models get stronger?

Compute cost: PPO requires 4 models in memory (policy, reference, value, reward). At 70B: 48 GPUs, 60 hours

Binary reward is information-poor: `71` on a problem with answer `72` gets the same 0 as `banana`

THE HUMAN SIDE

RLVR SHIFTS THE LABOR FRONTIER — BUT DOESN'T ELIMINATE HUMAN INVOLVEMENT

Even with RLVR, someone still **curates problems**, **writes verifiers**, and **builds benchmarks**. For everything in the "subjective" half of the spectrum, **human judgment remains irreplaceable**.

THE DEEPER CONNECTION

RLVR reduces dependence on human preference data for some tasks — but the preference data industry is *growing*, because the remaining tasks are harder and require more expertise. The workers producing this data are often the same professionals whose careers AI is disrupting.

Platforms such as **Mercor**, **Scale AI**, and **Surge AI** now hire lawyers, scientists, and writers as gig workers to produce this expert data — what *NY Magazine* called "*the largest harvesting of human expertise ever attempted.*"

Lan will now show us what that work actually looks like.

LIVE DEMO: RLVR TRAINING

Setup: OLMo-1B or Qwen2.5-0.5B · 200 GSM8K problems · exact-match verifier · Lambda A10

WHAT WE'LL DO

1. Evaluate the **base model** on 50 GSM8K test problems
2. Run RLVR training for a few hundred steps
3. Evaluate the **trained model** on the same problems
4. Compare outputs **side by side**

```
#### Before RLVR *"Natalia sold clips to 48 friends in April..."* "Natalia is a good person who sells clips. She has many friends. The weather in April is nice for selling clips..."
```

```
#### After RLVR *"Natalia sold clips to 48 friends in April..."* "In April, Natalia sold 48 clips. In May, she sold 48/2 = 24 clips. In total: 48 + 24 = **72 clips**."
```

Demo code in `demo/rlvr_demo.py`

NONE OF THIS IS MAGIC

Rewards. Gradients. Verifiers.

Nothing here is outside our toolkit. The frontier is open; the maths is ours.

WHAT COMES AFTER RLVR?

RLVR works because verification is cheap. The field is now asking: **what other axes can we scale?**

DIRECTION	KEY IDEA	WHO'S PUSHING IT
Inference-time compute	Let models "think longer" — a second scaling axis independent of model size	OpenAI (o3/o4), DeepSeek-R1
Self-play & autocurricula	Models generate their own tasks, solve, and verify — zero human data	Absolute Zero (NeurIPS '25 spotlight)
Process reward models	Score each reasoning *step*, not just the final answer — denser signal	OpenAI, ICLR '25 best paper
Agentic RL	Train in multi-turn, tool-using environments — optimise for task completion	Microsoft Research, Agent-R1
Diffusion LLMs	Generate all tokens in parallel, refine iteratively — 10× faster	Inception Labs (Mercury), Google

THE META-NARRATIVE

> *"Pre-training as we know it will unquestionably end … we have but one internet." — Ilya Sutskever, NeurIPS 2024*

The dominant paradigm is shifting from **scaling data** to **scaling algorithms**: smarter training signals, richer verification, and inference-time search. RLVR is the first step on that road.

CORE REFERENCES

Primary: Lambert et al. **"Tulu 3"** (arXiv:2411.15124, 2024) — blog: allenai.org/blog/tulu-3

Foundations: Ouyang et al. **"InstructGPT"** (NeurIPS 2022) · Schulman et al. **"PPO"** (2017) · Rafailov et al. **"DPO"** (NeurIPS 2024)

RL for Reasoning: Zelikman et al. **"STaR"** (NeurIPS 2022) · **"Quiet-STaR"** (COLM 2024) · Kazemnejad et al. **"VinePPO"** (2024)

RLVR at Scale: DeepSeek-AI **"DeepSeek-R1"** (arXiv:2501.12948, 2025)

Benchmarks: Cobbe et al. **"GSM8K"** (2021) · Hendrycks et al. **"MATH"** (2021) · Zhou et al. **"IFEval"** (2023)

Open Resources: `github.com/allenai/open-instruct` · `huggingface.co/allenai` · TRL: `github.com/huggingface/trl`